KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION (Deemed to be University)

CourseCompassAI: RAG Tutor for College Notes (StudyBuddy)

Project ID: Prj104
Team ID: T104

Team GitHub Repository: https://github.com/matlilakshmireddy63-droid/CourceCompassAI

Student Name	Registration Number	Individual GitHub Link
Matli Lakshmi Prasanna Kumar Reddy	99230040135	https://github.com/matlilakshmireddy63-droid/GEN-AI-MASTERY
Naru Mahesh Kumar Reddy	99230040146	https://github.com/narumaheshkumarreddy1-cmyk/GEN-AI
Narreddy Maheswar Reddy	99230040145	https://github.com/narumaheshkumarreddy1-cmyk/GEN-AI
D Sri Charan Rao	99230040099	https://github.com/Ravanagod/AI-GEN-MASTERY-ASSIGNMENTS

Project Trainer / Guide

Mr. SHIRWIN PRINCE I
Department of Computer Science and Engineering
Kalasalingam Academy of Research and Education

Table of Contents

1. Introduction 2
2. Problem Statement and Objectives 2
3. System Architecture and Flowcharts 3
4. Implementation Details and Technology Stack 6
5. Application Screens and Testing 7
6. Conclusion and Future Scope 9

1. Introduction

CourseCompassAI: RAG Tutor for College Notes is an AI-assisted academic learning application designed to help college students access, understand and organize their study materials. Students often refer to lecture notes, syllabi, textbooks and previous-year question papers while preparing for examinations. However, finding relevant information across multiple lengthy documents can be time-consuming. General-purpose chatbots may also provide answers that are not directly aligned with the student's course content.

To address these challenges, CourseCompassAI uses Retrieval-Augmented Generation (RAG), which combines document retrieval with AI-based response generation. Students can upload their academic documents, which are processed and indexed for efficient retrieval. When a student asks a question, the system searches the indexed course materials, retrieves relevant passages and provides them as context to a Large Language Model (LLM) to generate a contextual response. The application also includes features such as exam answer blueprints, interactive quizzes, course management and a study dashboard to support different academic activities.

By bringing these functionalities together in a single platform, CourseCompassAI aims to make learning more convenient, organized and interactive. Students can clarify doubts, understand difficult concepts, revise important topics and evaluate their knowledge using their own study materials. Rather than replacing traditional learning methods, the application serves as a supplementary academic assistant that helps students access relevant information, manage their learning resources and prepare for examinations at their own pace.

2. Problem Statement and Objectives

Problem statement: Students need a convenient way to search academic documents and obtain answers aligned with their selected course. Manual searching is slow, while answers from a general chatbot may not be supported by the student's notes or syllabus.

Objectives:

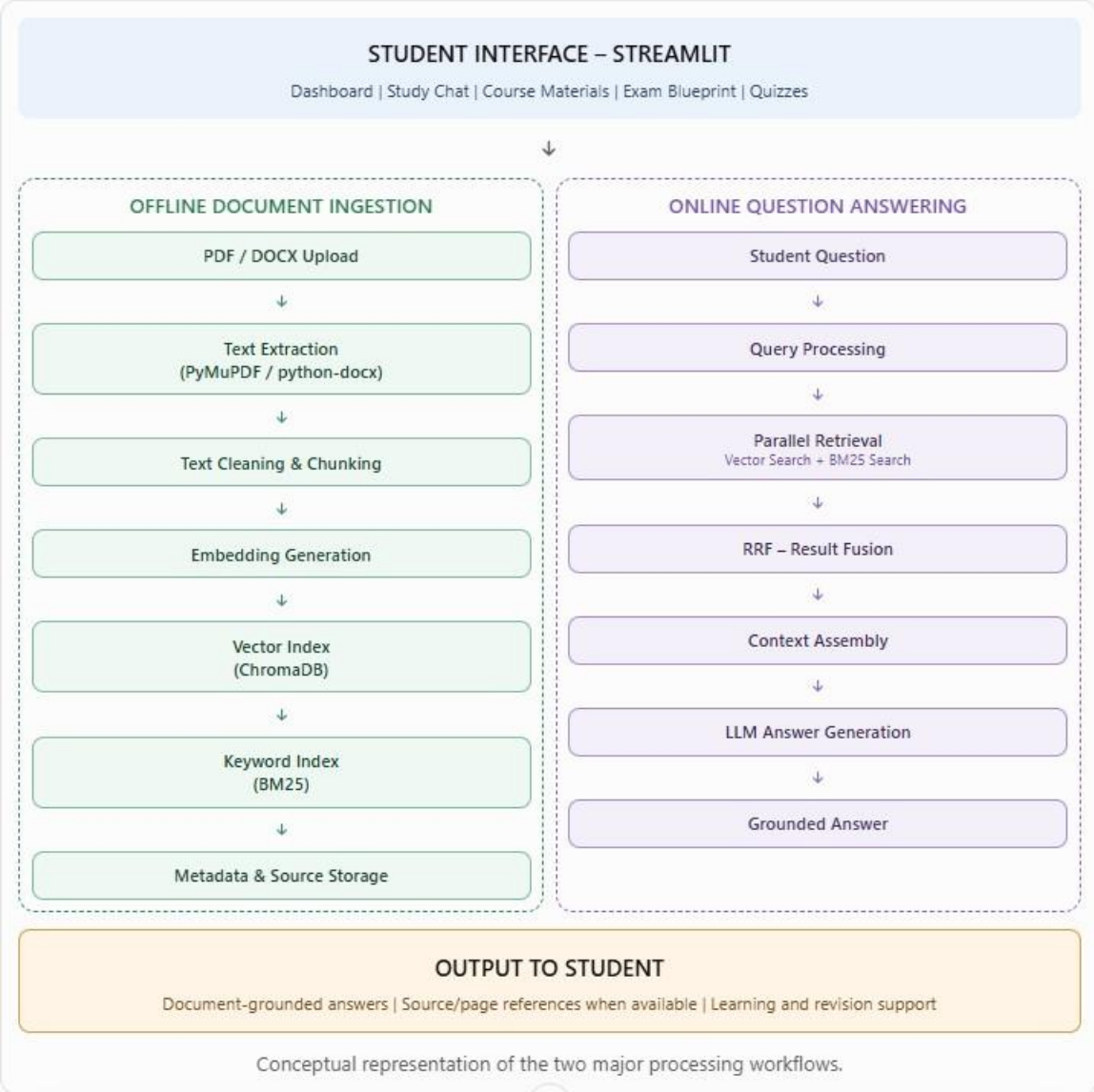
- Provide course-wise question answering over uploaded academic material.
- Extract PDF text and preserve source and page metadata.
- Combine semantic retrieval and keyword retrieval through ChromaDB and BM25.

- Use Reciprocal Rank Fusion (RRF) to combine retrieval rankings.
- Generate contextual answers with source references where available.
- Support exam preparation through 2-mark, 5-mark and 10-mark answer blueprints and interactive quizzes.

3. System Architecture and Flowcharts

3.1 End-to-End System Architecture

The application is organized around a Streamlit interface and two connected RAG workflows. The offline ingestion workflow prepares uploaded course documents for search. The online workflow processes a student's question, retrieves relevant passages and generates an answer using the configured LLM.



Grounded answer | Source/page citations | Exam blueprint | Quiz and revision support

Figure 3.1: End-to-end CourseCompass AI system architecture

3.2 Offline Document Ingestion Flowchart

The ingestion pipeline runs when course documents are added or indexed. Text is extracted page by page, divided into smaller chunks and enriched with metadata. Embeddings and keyword indexes are then created to support later retrieval.



Figure 3.2: Offline document ingestion and indexing flow

3.3 Online Query Retrieval and Answer Generation Flowchart

When a student asks a question, the system searches the indexed material in two ways. Dense vector search finds semantically related passages, while BM25 keyword search identifies lexical matches. RRF combines the rankings before the selected evidence is passed to the LLM.

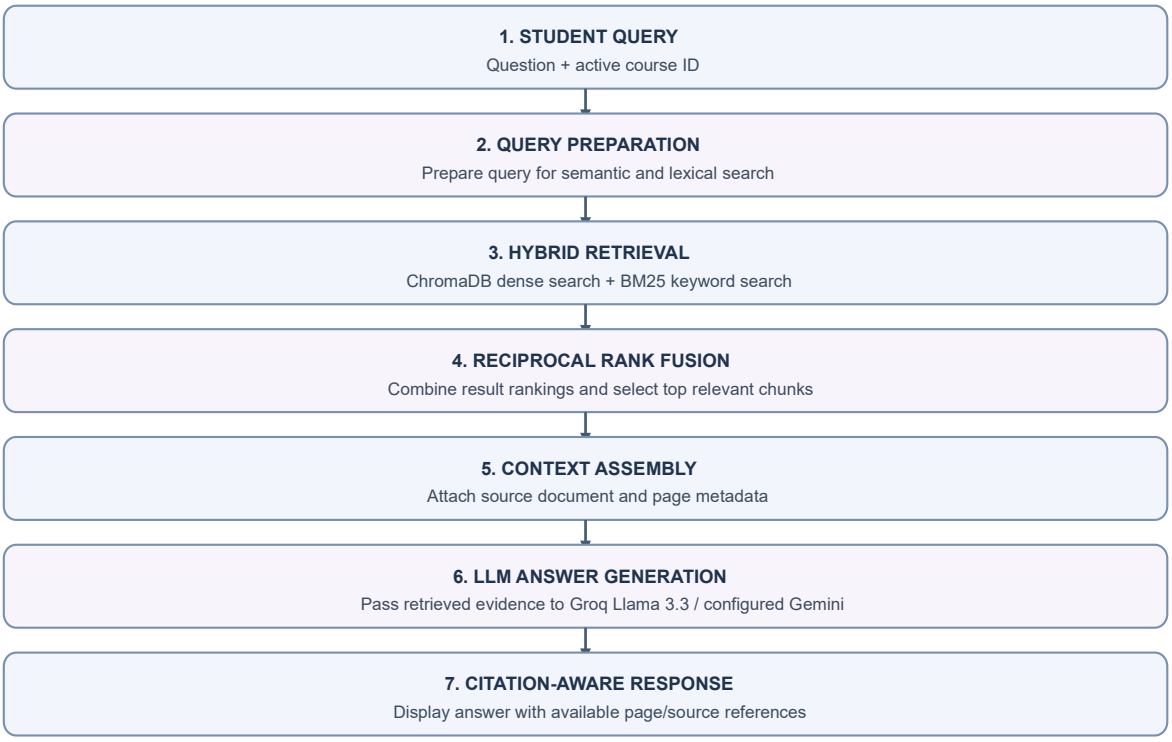


Figure 3.3: Online hybrid retrieval and answer generation flow

3.4 Student Authentication and Navigation Flowchart

The authentication interface provides student sign-in, account creation and Google sign-in options as shown in the application.

After a valid session is established, the student can access the dashboard and navigate to the study modules.

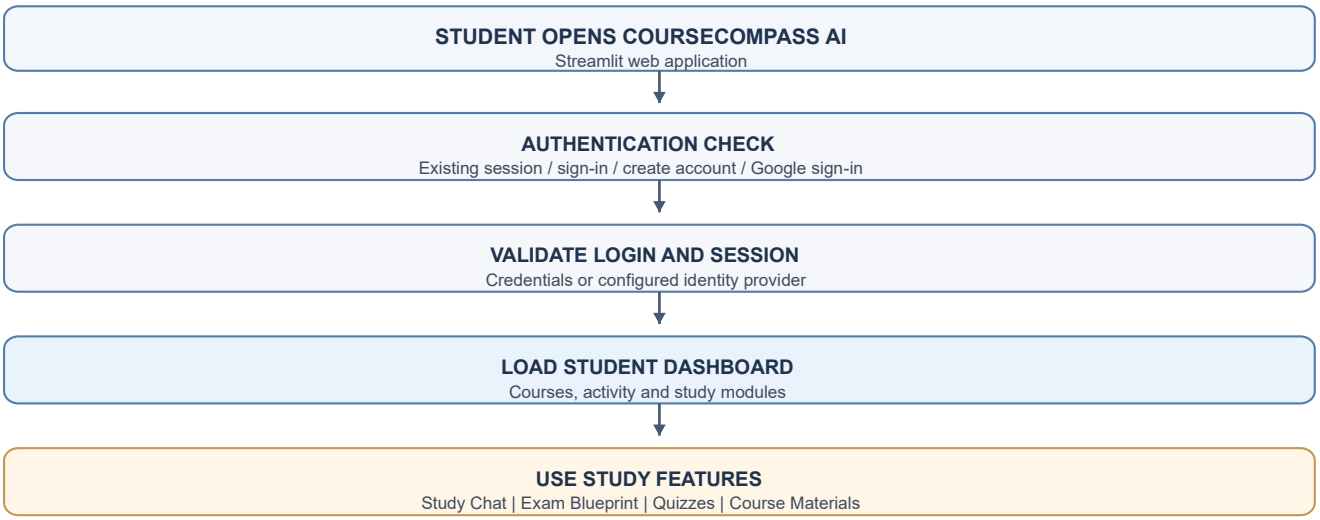


Figure 3.4: Student authentication and application navigation flow

4. Implementation Details and Technology Stack

Component	Technology	Purpose
Application interface	Streamlit	Dashboard, study chat, exam blueprint, quizzes and course materials
Programming language	Python	Application logic and RAG orchestration
PDF extraction	PyMuPDF; optional OCR / fallback parser	Extract text and retain page information
Chunking	Recursive text splitting	Divide documents into retrievable text segments
Embedding model	Sentence Transformers: all-MiniLM-L6-v2	Convert text chunks and queries into vectors
Vector search	ChromaDB	Semantic similarity retrieval
Keyword search	BM25 / Rank-BM25	Lexical matching of important terms
Rank fusion	Reciprocal Rank Fusion (RRF)	Combine dense and keyword result rankings
LLM inference	Groq Llama 3.3 / configured Gemini	Generate responses using retrieved context

4.1 Hybrid Retrieval

Hybrid retrieval combines dense semantic search and sparse keyword search. ChromaDB searches vectors created from the text embeddings. BM25 ranks text chunks according to keyword relevance. Reciprocal Rank Fusion combines the two ranked lists so that relevant passages found by either method can be considered during context selection.

4.2 Large Language Model and Grounded Responses

The retrieved text is assembled into a context and sent to the configured LLM for answer generation. The model is instructed to use the supplied evidence. The application can display source and page references associated with retrieved chunks. RAG helps ground responses in the uploaded material, but it does not guarantee that every generated answer is error-free.

5. Application Screens and Testing

The supplied website screenshots show the CourseCompass AI sign-in page, academic dashboard, University Exam Answer Blueprint and Quiz & Exam Preparation Studio. The dashboard displays enrolled courses, document counts, indexed chunks and recent activity. The Exam Blueprint screen allows students to enter a question and choose a 2-mark, 5-mark or 10-mark answer format. The quiz screen allows topic selection, question count and assessment focus.

Functional testing areas:

- Sign-in, account creation and session handling.
- Course selection and navigation between modules.
- Uploading PDFs and extracting page-level text.
- Chunk creation and indexing in the retrieval stores.
- Dense and BM25 retrieval, followed by RRF fusion.
- Answer generation and source/page citation display.
- Exam blueprint and quiz generation.

Evidence note: Screenshots supplied for the dashboard show zero uploaded documents and zero indexed chunks at the time of capture. Therefore, the screenshots demonstrate the interface and available modules, but they do not by themselves establish successful end-to-end retrieval accuracy or performance measurements.

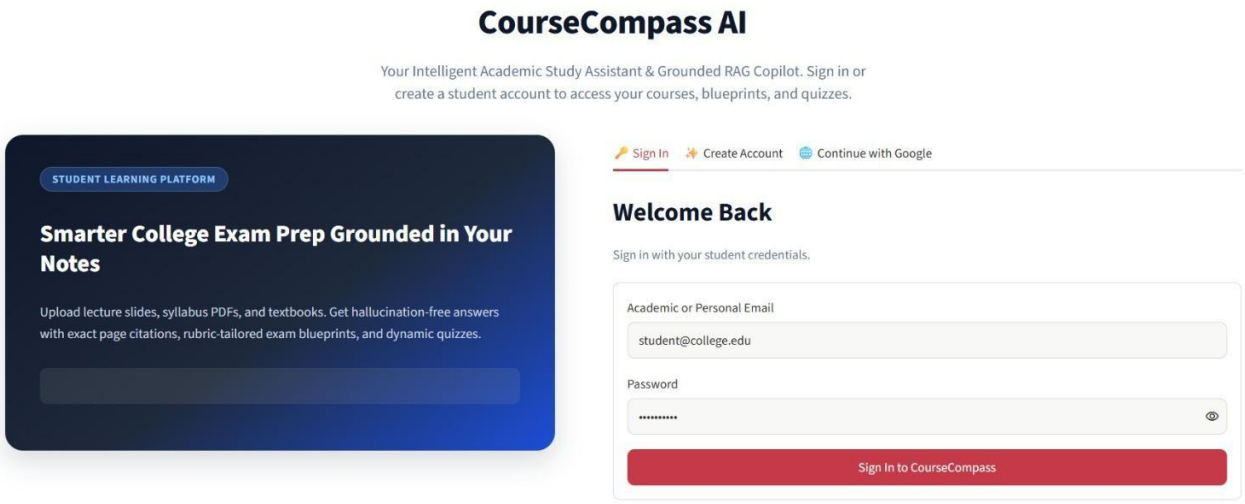


Figure 5.1 Login page screenshot

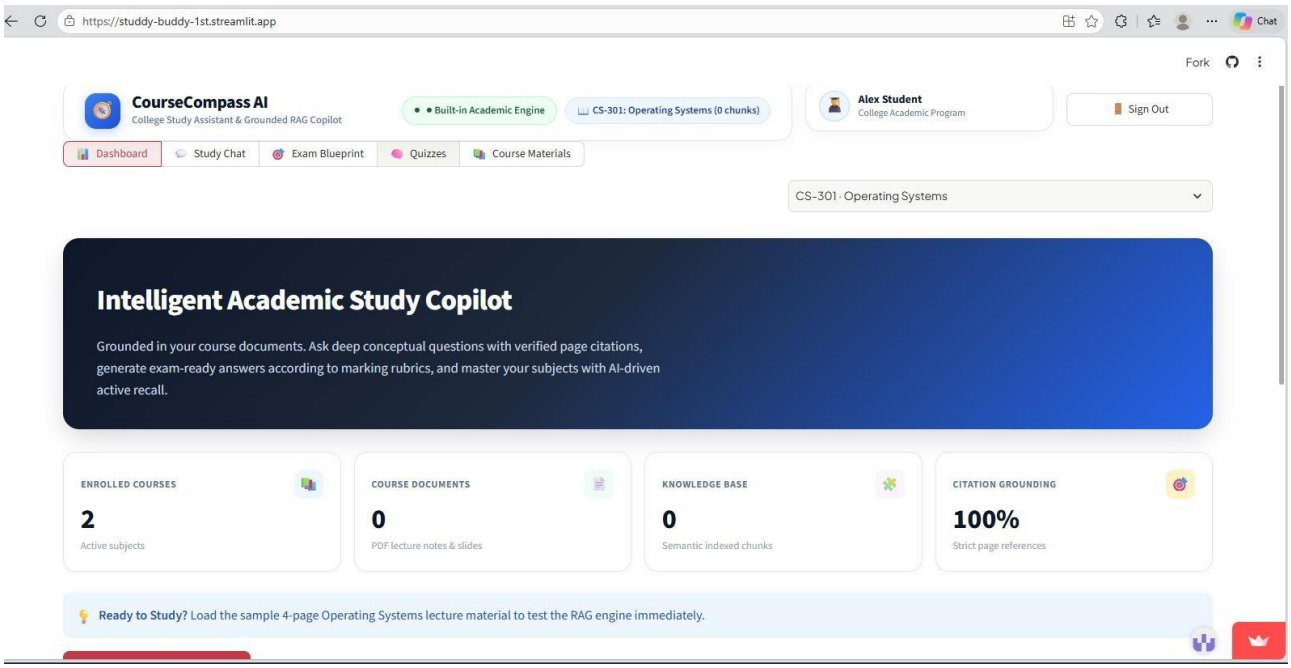


Figure 5.2: Dashboard Screenshot-1

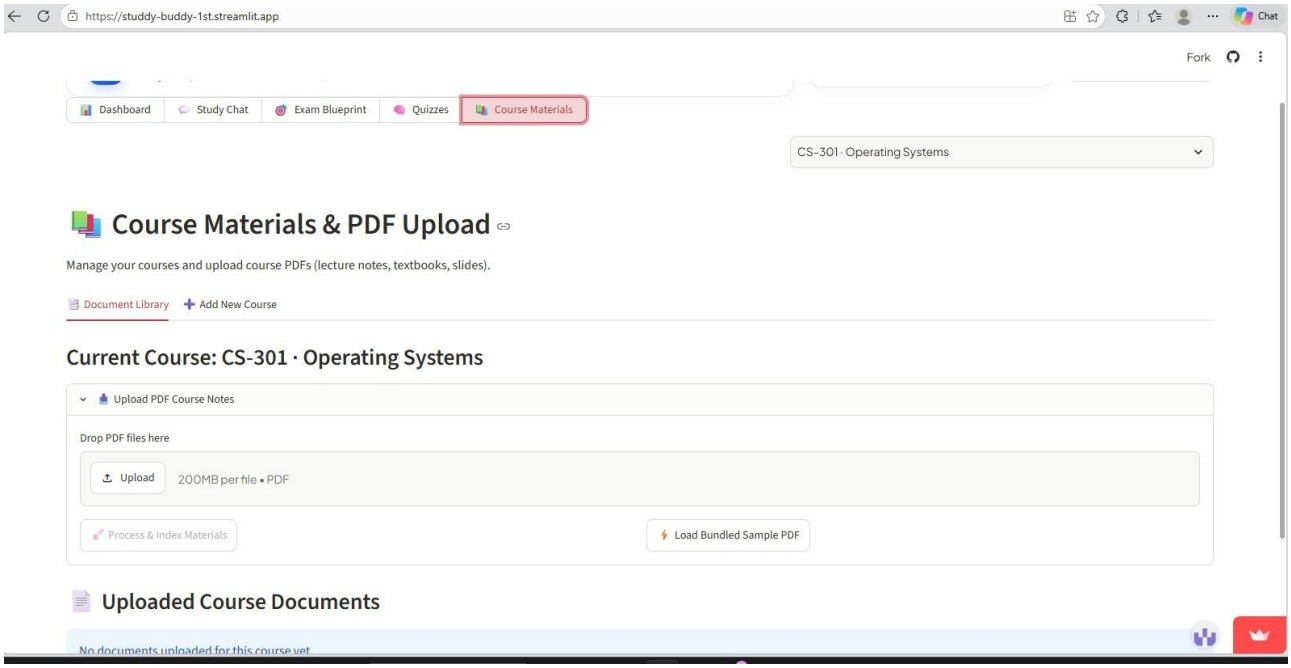


Figure 5.3: Dashboard Screenshot-2

6. Conclusion and Future Scope

Conclusion: CourseCompassAI: Rag Tutor for College Notes provides an academic assistant that connects student questions with course documents through a RAG workflow. The design combines PDF extraction, text chunking, embeddings, ChromaDB vector retrieval, BM25 keyword search and Reciprocal Rank Fusion. Retrieved context is supplied to an LLM to generate a response, with source metadata supporting verification. The Streamlit application brings together the dashboard, study chat, exam answer blueprint, quizzes and course materials.

Future scope: Possible enhancements include stronger OCR for scanned and handwritten notes, improved extraction of tables and diagrams, expanded support for more document formats, deeper previous-year question trend analysis, personalized revision recommendations, downloadable study notes and additional evaluation of retrieval relevance, answer faithfulness and response time. Statistical prediction of likely exam topics should be treated as a future module unless it is separately implemented and tested.

Project live link: https://studdy-buddy-1st.streamlit.app/